

Reviewer Pack — Free Cumulant Norm Gap in Read-Twice BPs for IP_2

Entry 3a65c68aefcb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 01:46:57 UTC

Free Cumulant Norm Gap in Read-Twice BPs for IP_2

Entry ID: 3a65c68aefcb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 01:46:57 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs for IP_2

Statement:

For any read-twice BP P computing IP_2 on n variables, the operator norm of its free cumulant transform satisfies $\|\rho(P)\| = \Omega(n)$, while for random read-twice BPs (not computing IP_2), $\|\rho(P)\| = O(\log n)$.

Rationale (proposer's reasoning):

Free cumulants capture non-commutative independence structures; IP_2's read-twice BP requires entangled correlations that inflate the cumulant norm, while random BPs exhibit asymptotic freeness with trivial norm scaling.

Taxonomy category: BP_READTWICE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c3f586b91ac8ec84

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    n = 40
    random.seed(seed)

    def generate_transition_matrix():
        return [[random.random() for _ in range(n)] for _ in range(n)]

    def matrix_multiply(A, B):
        result = [[sum(a * b for a, b in zip(row_a, col_b)) for col_b in zip(*B)] for row_a in A]
        return result

    def r_transform(P):
        I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        P_inv = [row[:] for row in P]
        for _ in range(1, n):
            P_inv = matrix_multiply(P_inv, P)
        R = [[P[i][j] - I[i][j] for j in range(n)] for i in range(n)]
        return R

    def operator_norm(R):
        max_eigenvalue = 0
        for _ in range(100): # Power iteration method
            v = [random.random() for _ in range(n)]
            v = [x / math.sqrt(sum(x**2 for x in v)) for x in v]
            Rv = matrix_multiply(R, v)
            max_eigenvalue = max(max_eigenvalue, abs(v[0] * Rv[0][0]))
        return max_eigenvalue

    # Generate IP_2's read-twice BP
    P_ip2 = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    P_ip2[0][1], P_ip2[1][0] = 1, 1
    R_ip2 = r_transform(P_ip2)
    norm_ip2 = operator_norm(R_ip2)

    # Generate 30 random read-twice BPs
    results = []
    for _ in range(30):
        P_random = generate_transition_matrix()
        R_random = r_transform(P_random)
        norm_random = operator_norm(R_random)

        if norm_random > 1.2 * math.log(n):
```

```

        counterexample = "Random BP with high norm"
        return {
            "metric_name": "Operator Norm",
            "metric_value": norm_random,
            "instances_tested": 30,
            "conjecture_holds": False,
            "counterexample": counterexample
        }

    results.append(norm_random)

    if norm_ip2 < 0.9 * n:
        counterexample = "IP_2 BP with low norm"
        return {
            "metric_name": "Operator Norm",
            "metric_value": norm_ip2,
            "instances_tested": 30,
            "conjecture_holds": False,
            "counterexample": counterexample
        }

    return {
        "metric_name": "Operator Norm",
        "metric_value": norm_ip2,
        "instances_tested": 30,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= 0.9 * n) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = seeds[next(i for i, r in enumerate(results) if r < 0.9 * n)]
        print(f"RESULT: FALSIFIED counterexample=\"IP_2 BP with low norm\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cafc56ca.py", line 95, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cafc56ca.py", line 50, in run_trial
    norm_ip2 = operator_norm(R_ip2)
                ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cafc56ca.py", line 42, in operator_norm
    Rv = matrix_multiply(R, v)
          ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cafc56ca.py", line 26, in matrix_multiply
    result = [[sum(a * b for a, b in zip(row_a, col_b)) for col_b in zip(*B)] for row_a in A]
               ^^^^^^^
```

TypeError: 'float' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with TypeError during matrix multiplication, preventing result collection |
next: Debug matrix_multiply function's handling of float vs iterable types

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	41336
2	preregistration	ollama_remote	qwen3:8b	0	0	27636
3	novelty	ollama_remote	qwen3:8b	0	0	20917
4	novelty	ollama_remote	qwen3:8b	0	0	13338
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16657
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10171
7	judge	ollama_remote	qwen3:8b	0	0	20946

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 151002 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/3a65c68aefcb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/3a65c68aefcb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3a65c68aefcb.tar.gz` (if generated)